

Self-Supervised Learning of Parametric Cutting Planes for Mixed-Integer Nonlinear Programs through Exact Differentiable Optimization Layers

Parth Brahmabhatt
University of Wisconsin–Madison
prbrahmabhatt@wisc.edu

Abstract

Many online decision problems are parametric mixed-integer nonlinear programs (MINLPs) that must be re-solved every time their inputs change, yet are too expensive to solve exactly in real time. A promising surrogate is a convex relaxation tightened by *learned, parametric cutting planes*: a model maps the instance parameters to linear inequalities, and the cut relaxation is solved instead of the MINLP. Dixit, Gupta and Zhang [12] learned such cuts for mixed-integer *linear* programs by solving a large bilevel program whose lower levels are replaced by their KKT conditions, supervised by the optimal solutions of the original problem. We instead place the cut relaxation *inside* a neural network as an exact differentiable conic optimization layer and train the cut generator by stochastic gradient descent on the cost of the solution it ultimately deploys. Training is *self-supervised*: no optimal solutions are computed for training or model selection. Making this work required (i) a two-layer differentiable deployment map (relaxation $\rightarrow$ soft rounding $\rightarrow$ continuous re-solve) whose gradient is exact rather than smoothed, (ii) a reverse-mode conic layer that remains accurate when the KKT system is singular, a regime in which `cvxpylayers` returns a 97%-wrong gradient on our problems, and (iii) a depth-anchored cut parameterization that keeps every cut active, and therefore trainable, from initialization. For nonconvex problems the same network also predicts the linearization point of the convexification. We evaluate on AC unit commitment on the IEEE 118- and 300-bus systems (54 and 69 binaries, nonconvex AC physics) and on a convex hybrid-vehicle MINLP with 30 integer modes. The learned surrogate deploys in 3.3 s, 31.9 s and 0.011 s per instance, $24\times$, $11\times$ and $92\times$ faster than the reference solvers, with mean cost deviations of 0.176%, 0.061% and 0.123%. Against a supervised neural-network proxy that is given the reference solutions and the same feasibility restoration, the learned cut is $15\times$ more accurate in cost on the vehicle problem; on AC unit commitment the proxy is more accurate on case118, while on case300 the learned cut attains the lower cost deviation (0.061% vs. 0.075%) and the proxy the lower decision errors. Our method uses none of the 4.0 h (case118) and 13.2 h (case300) of reference solves that the proxy is trained on.

1 Introduction

Unit commitment, energy management and hybrid-powertrain control are solved repeatedly, every few minutes or seconds, with only some parameters changing between solves: loads, prices, driver demand. Each solve is a mixed-integer program, often with nonlinear physics, and a global solve can take minutes. This has motivated a large body of work on *learning to optimize*: training a model offline so that, online, a solution is produced in a fraction of the solver time [5, 21].

The most common approach is an *optimization proxy*, a neural network that maps the parameters directly to a solution, trained either on solver outputs [7, 15] or self-supervised on the objective with penalties for violated constraints [9, 13, 23, 26]. A proxy has to rediscover the problem’s constraints from data and usually needs a restoration step to become feasible. A different idea keeps the optimization model and learns only what it lacks. Dixit, Gupta and Zhang [12] observe that the linear relaxation of a MILP becomes exact once the right inequalities are added, and that these inequalities can be *learned as functions of the parameters*: a decision-focused surrogate model, $\min\{c^\top x : Ax \leq b, \tilde{A}(u; \theta)x \leq \tilde{b}(u; \theta)\}$. The surrogate keeps every original constraint, and the learned rows act as parametric cutting planes generated before the instance arrives.

How the cuts are learned matters. In [12] the training problem is a bilevel program: the upper level minimizes $\sum_i \|x_i^* - \tilde{x}_i\|_1$, the distance between the MILP optimum x_i^* and the surrogate solution $\tilde{x}_i$; each lower level is the

surrogate LP for instance i . The lower levels are replaced by their KKT conditions, complementarity is handled by an exact penalty, and the resulting multiconvex problem is solved by block-coordinate descent [18]. This is accurate and data-efficient, but it ties the method to (a) labels x_i^* , one global MINLP solve per training point, (b) a lower level whose KKT system can be written in closed form, here an LP, and (c) a single monolithic optimization over all training points, about 10^4 s for 100 points with polynomial cut coefficients [12].

This paper. We keep the idea of learned parametric cuts and change how they are learned. The cut relaxation becomes a *differentiable optimization layer* [1, 3], the cut generator becomes a neural network, and the bilevel program is replaced by stochastic gradient descent through the layer. Because the gradient of any downstream loss is available, the loss no longer has to be distance to a label: we train on the *cost of the solution that is actually deployed*, after rounding and a continuous re-solve. Training is therefore self-supervised. Reference solutions are used only to score held-out test instances. Our contributions are:

1. **Learned cuts through an exact differentiable layer, without labels.** A network maps parameters to cut rows of two categories, mixed continuous–integer and integer-only. It is trained end-to-end on deployed cost through two chained conic layers with a differentiable rounding stage between them (Section 4). The method extends from MILP/LP surrogates to MINLPs with second-order-cone relaxations, and for nonconvex problems it also learns the point at which the problem is convexified.
2. **Exact gradients when the KKT system is singular.** On AC unit commitment the relaxation violates LICQ (rank deficiency 345, KKT condition number $\sim 10^{21}$). Krylov solvers used by `diffcp/cvxpylayers` [2] stall and return a 97%-wrong gradient. A direct sparse LU of the regularized KKT matrix recovers the primal derivative (cosine 0.99 against converged finite differences) in 0.01 s (Section 4.3).
3. **Exact beats smoothed.** With one solve, the exact layer’s gradient has cosine 0.9996 with the truth; a perturbation (smoothing) estimator [6] reaches 0.41 at 4 solves and 0.82 at 64 (Section 5.3).
4. **Three case studies,** two nonconvex AC unit-commitment systems and one convex vehicle-control MINLP, compared with a supervised proxy and with the reference solvers on discrete error, continuous error, cost and solution time (Section 5).

2 Related work

Learning cuts and surrogate models. Learning *which* cuts a branch-and-cut solver should add has been studied with reinforcement and imitation learning [24, 27]; there the cuts are generated by the solver and only selected by the model. Decision-focused surrogate modeling [12] instead learns the cut coefficients themselves as parametric functions, so that a single LP replaces the MILP online; its training problem is the bilevel program described above, solved by penalty block-coordinate descent [18]. We share its premise and surrogate form and replace its learning machinery; Table 1 summarizes the differences. Related learned parametric approximations of power-system optimization have been trained self-supervised [4], and convexified layers have been used for neuro-symbolic learning [22].

Differentiable optimization. OptNet [3] differentiates QPs through their KKT system; `cvxpylayers` [1] extends this to disciplined parametrized convex programs via the cone-program derivative of `diffcp` [2]; DiffOpt.jl [8] does the same in JuMP. Predict-then-optimize methods [14, 25, 30] differentiate (surrogates of) combinatorial solvers, often via smoothing [6, 29]. These works assume, implicitly or explicitly, a nondegenerate solution; implicit-function bilevel learning shares the assumption [17]. Our relaxations violate it, and Section 4.3 shows what this does to off-the-shelf layers.

Learning to optimize MINLPs. L2O for parametric MINLPs [26] and differentiable predictive control with integer inputs [9] train a network to output integer and continuous decisions directly, with differentiable rounding and a penalty or projection for feasibility. Our network never outputs a decision; it outputs constraints, and the decision comes from an optimization solver.

Table 1: Learning parametric cuts: decision-focused surrogate modeling (DFSM) [12] versus this work.

	DFSM [12]	This work
Original problem	MILP	MINLP (convex or nonconvex)
Surrogate	LP with learned cuts	SOCP relaxation with learned cuts (+ learned linearization point)
Cut model	low-order polynomial in u	neural network
Training signal	$\sum_i \ x_i^* - \tilde{x}_i\ _1$ (supervised)	deployed cost (self-supervised)
Labels needed	optimal x_i^* for every training point	none
Learning algorithm	KKT reformulation, exact penalty, BCD	SGD through an exact differentiable conic layer
Training cost	$\sim 10^4$ s for 100 points	$1.4\text{--}6.3 \times 10^3$ s for 144 points (one core)

3 Problem setting

We consider a family of MINLPs indexed by parameters $\xi \in \Xi$,

$$J^*(\xi) = \min_{x \in \mathbb{R}^n, z \in \mathbb{Z}^m} f(x, z; \xi) \quad \text{s.t.} \quad (x, z) \in \mathcal{F}(\xi), \quad z \in \{0, \dots, S\}^m, \quad (1)$$

with x continuous and z integer (binary when $S = 1$). Let $\mathcal{C}(\xi; V)$ be a convex set obtained by relaxing integrality and, when $\mathcal{F}$ is nonconvex, replacing the nonconvex constraints by a convexification around a point V (Section 4.4). The *learned surrogate* adds R cut rows produced by a network h_ϕ :

$$(\hat{x}, \hat{z})(\xi; \phi) \in \arg \min_{x, z} \left\{ f(x, z; \xi) : (x, z) \in \mathcal{C}(\xi; V_\phi(\xi)), \quad A_\phi(\xi) w(x, z) \leq b_\phi(\xi) \right\}, \quad (2)$$

where $w(x, z)$ stacks the variables the cuts act on and $(V_\phi, A_\phi, b_\phi) = h_\phi(\xi)$. Problem (2) is a conic program (a second-order cone program in all our cases) and is solved online in place of (1). Its integer block $\hat{z}$ is generally fractional; a *deployment map* turns it into a feasible solution: rounding, a feasibility restoration Π , and a continuous re-solve with z fixed,

$$\bar{z} = \Pi(\hat{z}; \xi) \in \{0, \dots, S\}^m, \quad \bar{x} \in \arg \min_x \{ f(x, \bar{z}; \xi) : (x, \bar{z}) \in \mathcal{F}(\xi) \}, \quad J_{\text{dep}}(\xi; \phi) = f(\bar{x}, \bar{z}; \xi). \quad (3)$$

The goal is to choose ϕ so that J_{dep} is close to J^* , using only parameter samples $\{\xi_i\}$ and *not* their solutions.

4 Method

4.1 Cut parameterization and depth anchoring

The network emits K rows in each of two categories:

$$\text{general:} \quad \alpha_k^\top x_{\mathcal{I}} + a_k^\top z \leq b_k, \quad k = 1, \dots, K, \quad (4)$$

$$\text{integer-only:} \quad a_k'^\top z \leq b'_k, \quad k = 1, \dots, K, \quad (5)$$

where $x_{\mathcal{I}}$ is the continuous block that interacts with the integers (generator outputs in unit commitment; the energy, engine-power and battery-power trajectories in the vehicle problem). Coefficients are bounded, $a = 1 + 2 \tanh(\cdot)$ and $\alpha = 2 \tanh(\cdot)$, with the final layer initialized at zero so that training starts from a cardinality-type cut $\mathbf{1}^\top z \leq b$; a small random spread between rows avoids duplicate constraints.

The right-hand side is *not* predicted directly. A cut that is slack at the surrogate optimum has zero derivative with respect to its own coefficients (Proposition 1), so a row that starts slack never trains; in early experiments two of four directly-predicted rows started with slack 0.23 and 0.54 and stayed there. We therefore predict a depth $d_k = d_{\max} \sigma(\cdot) \in (0, d_{\max})$ and anchor

$$b_k = \langle A_k, \bar{w}(\xi) \rangle - d_k, \quad (6)$$

where $\bar{w}(\xi)$ is the solution of the uncut relaxation, a label-free quantity. Every row then cuts off $\bar{w}$ by exactly d_k , and in our runs all rows were active at initialization. The anchor is assembled inside the autodiff graph, so the gradient with respect to A_k includes the path through b_k .

Proposition 1 (slack cuts do not learn). *Let (x^*, y^*) be a primal-dual solution of (2) with strict complementarity at which row k is slack, $A_k w(x^*) < b_k$. Then $y_k^* = 0$, the solution is locally constant in (A_k, b_k) , and $\partial L / \partial (A_k, b_k) = 0$ for any loss L of the solution.*

Sketch. Under strict complementarity the active set is locally constant, so row k can be deleted without changing the solution in a neighbourhood of (A_k, b_k) ; the implicit-function derivative of x^* with respect to its data is then zero. $\square$

Cuts are imposed *hard*. A softly priced cut $A_k w \leq b_k + s_k$, $s_k \geq 0$, with linear price κs_k is tempting because it can never make the surrogate infeasible, but once violated its right-hand side drops out of the optimality conditions: the penalty gradient is the constant κ . We observed exactly this (a learned band violated on all instances could be replaced by an arbitrary one with bit-identical results). We fall back to the priced form only when the hard cut makes an instance infeasible, so a bad cut costs rather than crashes.

4.2 Self-supervised deployed-cost loss

The deployment map (3) is piecewise constant in $\hat{z}$: rounding and restoration make it a staircase (on case118 the deployed cost was flat to the cent over perturbations of 0.1 in $\hat{z}$, then jumped), so its true gradient is zero almost everywhere. We train on a differentiable version of it, two chained conic layers:

$$\xi \xrightarrow{h_\phi} (V, A, b) \xrightarrow{\text{layer 1: (2)}} \hat{z} \xrightarrow{\rho_\tau} \tilde{z} \xrightarrow{\text{layer 2: re-solve at } z=\tilde{z}} (\tilde{x}, \tilde{z}) \rightarrow \ell = \frac{f(\tilde{x}, \tilde{z}; \xi)}{J_R(\xi)}. \quad (7)$$

Layer 2 is the same compiled conic program with the bounds on z pinned to $\tilde{z}$; it prices the soft-rounded commitment. The normalizer $J_R(\xi)$ is the optimal value of the uncut relaxation, so the loss is scale-free across instances without using J^* . For unit commitment we add $\mu (\max\{0, r(\xi) - \bar{p}^\top \tilde{z}\} / J_R)^2$ for reserve shortfall; for the vehicle problem we add an integrality pressure $w_{\text{int}} \sum_t \sin^2(\pi \hat{z}_t) / J_R$ on the relaxed integers. Restoration loops are kept out of the training map. They can only lower cost or restore feasibility, so training optimizes an upper bound on deployed cost. Model selection (best epoch) uses the same loss on a held-out validation split. No step of training or selection uses J^* , x^* or z^* .

Soft rounding. For binaries, $\rho_\tau(\hat{z}) = \epsilon + (1 - 2\epsilon) \sigma((\hat{z} - t)/\tau)$ with $\tau = 0.08$; the affine squash with $\epsilon = 0.02$ (rather than clipping) keeps $\tilde{z}$ strictly interior for layer 2 without zeroing derivatives. For integers in $\{0, \dots, S\}$ the hard rule $z = \sum_{k=0}^{S-1} \mathbb{1}_{\{\hat{z} > k+t\}}$ becomes a sum of S sigmoids. In unit commitment the threshold is lowered until the soft commitment meets the reserve requirement, $t = \max\{t' \leq t_0 : \bar{p}^\top \rho_\tau(\hat{z}; t') \geq r\}$, computed by bisection. Since t then depends on $\hat{z}$, the implicit function theorem adds a rank-one term to the rounding Jacobian. With $\delta_j = \partial \tilde{z}_j / \partial (\hat{z}_j - t)$, the vector-Jacobian product is

$$\frac{\partial \ell}{\partial \tilde{z}} = \delta \odot g - \frac{g^\top \delta}{\bar{p}^\top \delta} (\bar{p} \odot \delta), \quad g = \frac{\partial \ell}{\partial \tilde{z}}. \quad (8)$$

Omitting the second term changes this link's cosine with finite differences from 1.000000 to 0.9955.

4.3 Exact differentiation of degenerate conic layers

Each layer is a DPP-compliant `cvxpy` problem [11] canonicalized to $\min_x \frac{1}{2} x^\top P x + q^\top x$ s.t. $Ax + s = b$, $s \in \mathcal{K}$, solved by Clarabel [16]. With $w = s - y$, $s = \Pi_{\mathcal{K}}(w)$, the optimality conditions read $F(x, w; \theta) = 0$ with $F_1 = Px + q + A^\top (\Pi_{\mathcal{K}}(w) - w)$ and $F_2 = Ax + \Pi_{\mathcal{K}}(w) - b$, and their Jacobian is

$$\mathsf{K} = \begin{bmatrix} P & A^\top (D\Pi_{\mathcal{K}}(w) - I) \\ A & D\Pi_{\mathcal{K}}(w) \end{bmatrix}. \quad (9)$$

For a scalar loss, one transposed solve $\mathsf{K}^\top g = [\partial \ell / \partial x; 0]$ gives every parameter derivative as $\partial \ell / \partial \theta_i = g_1^\top (-dq_i - dP_i x - dA_i^\top y) + g_2^\top (db_i - dA_i x)$. Because the problem is DPP, the canonical data are affine in θ , so the directions (dP_i, dq_i, dA_i, db_i) are constant: we extract them once and reuse them for every instance and step. The same layer differentiates the SOCP of unit commitment and the convex quadratically-constrained program of the vehicle problem.

Table 2: Solving the singular KKT system (9) on case118. Cosine of $\partial\ell/\partial b$ against converged central finite differences.

Linear solve	cosine	time
plain sparse LU	fails (singular / NaN)	–
LSQR / LSMR (as in <code>diffcp</code>), any damping	0.18	8.4 s
truncated-SVD pseudo-inverse	0.984	81 s
direct LU on $K + \varepsilon I$	0.987	0.01 s

Degeneracy. At the case118 surrogate optimum, 2154 rows are active but they have rank 1809, a deficiency of 345 with a clean spectral gap, and K has condition number $\sim 10^{21}$. The causes are structural. Where a linearization residual is exactly zero, both sides of $|r| \leq \xi$ and $\xi \geq 0$ are active on one degree of freedom. Where $u = 1$, the bounds $p \leq \bar{p}$ and $p \leq \bar{p}u$ coincide. Strict complementarity holds (no nonnegative pair has both s and y near zero), so the failure is LICQ alone. The null space of K lies almost entirely in the dual block (the x -component of the smallest singular vectors is ≈ 0), so $\partial x/\partial\theta$ is unique even though $\partial y/\partial\theta$ is not. Damping the dual directions should therefore leave the primal derivative intact, and Table 2 confirms it: a direct LU of $K + \varepsilon I$ is as accurate as a truncated-SVD pseudo-inverse and $8000\times$ faster, while LSQR/LSMR, the solvers behind `diffcp`, stall at cosine 0.18 for every damping value. Through `cvxpylayers` the gradient on this problem is 97% wrong. On a small nondegenerate SOCP both methods agree with finite differences (relative errors 1.5×10^{-8} ours, 5.0×10^{-8} `cvxpylayers`). We choose ε on a validation split of instances and report held-out cosines; the plateau is broad (10^{-6} to 10^{-9} all exceed 0.993 on validation). Repeated sparse factorizations occasionally abort the process inside SuperLU, so factorizations run in a persistent helper process that is restarted, with a small diagonal jitter, when it dies.

4.4 Learning where to convexify (nonconvex problems)

For AC unit commitment, $\mathcal{F}$ contains the nonconvex AC power-flow equations. We use the quadratically-constrained convex approximation (QCAC) of Constante-Flores and Li [10]. In rectangular voltages (v^r, v^i) , the products $c_{ii} = |v_i|^2$, c_{ij} and s_{ij} are linearized around a base point $V = (V^r, V^i)$, the linearization residual is absorbed by a slack $\xi \geq 0$ priced at $\rho = 10^6$, and an SOC constraint $\|(2c_{ij}, 2s_{ij}, c_{ii} - c_{jj})\| \leq c_{ii} + c_{jj}$ strengthens the result [20]. Branch limits are SOC constraints. With $u \in [0, 1]^G$ this is the convex set $\mathcal{C}(\xi; V)$; its quality depends on V . The QCAC method iterates: solve at V , re-linearize at the solution, and escalate ρ until the slack vanishes. We instead let the network predict $V_\phi(\xi) = (1 + 0.5 \tanh(\cdot), 0.5 \tanh(\cdot))$ and differentiate through it. V enters both layers, five parameter blocks of which three are nonlinear in V (e.g. $|V_i|^2$ and $V_i^r V_j^r + V_i^i V_j^i$), and the chain rule back to V is written out explicitly. Learning V by the exact gradient trains; learning it with a smoothed estimator made the training loss *rise* on both systems.

4.5 Deployment

Online, the method costs one network forward pass, one conic solve of (2), and the restoration Π . The supervised proxy baseline goes through the same restoration.

Unit commitment. Round $\hat{u}$ at threshold t . Commit additional units in a priority order π until the reserve $\bar{p}^\top u \geq 1.1 \sum p_d$ holds. Price the commitment, i.e. re-solve the QCAC with u fixed, re-linearizing at the solution until the slack is $\leq 10^{-6}$ (an AC-feasible operating point), and switch on the next unit in π until pricing is feasible. Then try switching off up to N_{down} committed units in reverse order of π , keeping each change that lowers cost. On case118, π is the network’s confidence (decreasing $\hat{u}$), $N_{\text{down}} = 25$, and t is a per-generator vector $t_k = 0.5 + c_0 + c_1 z(\bar{p}_k) + c_2 z(\text{mc}_k) + c_3 z(\text{nl}_k)$ whose four coefficients are fitted by random search on the *training* split’s deployed cost normalized by J_R (label-free). On case300, π is merit order (increasing average cost at full output), $N_{\text{down}} = 40$, and $t = 0.8$. Training on case300 rounds at the same t . On case118 it rounds at 0.5.

Vehicle. The integer schedule is the ℓ_1 -nearest integer point to $\hat{z}$ that is feasible for (1), $\min \|z - \hat{z}\|_1$ s.t. $(x, z) \in \mathcal{F}$, the projection-based restoration of [12], followed by the continuous re-solve.

Algorithm 1 Self-supervised training of the cut generator

Require: parameter samples $\{\xi_i\}$ (no solutions), network h_ϕ , compiled conic program

- 1: precompute uncut relaxation solutions $\bar{w}(\xi_i)$ and values $J_R(\xi_i)$
 - 2: **for** each minibatch $\mathcal{B}$ **do**
 - 3: **for** $i \in \mathcal{B}$ **do**
 - 4: $(V, A', d) \leftarrow h_\phi(\xi_i)$; $b \leftarrow \langle A', \bar{w}(\xi_i) \rangle - d$ $\triangleright$ depth anchoring (6)
 - 5: $\hat{z} \leftarrow$ layer 1 at (V, A', b) ; $\tilde{z} \leftarrow \rho_\tau(\hat{z})$ $\triangleright$ reserve-aware soft rounding
 - 6: $(\tilde{x}, \tilde{z}) \leftarrow$ layer 2 at $(V, \tilde{z})$; $\ell_i \leftarrow f(\tilde{x}, \tilde{z})/J_R(\xi_i)$ (+ penalties)
 - 7: $\partial \ell_i / \partial (V, A', b)$ by two reverse-mode KKT solves and (8)
 - 8: **end for**
 - 9: backpropagate $\sum_i \partial \ell_i / \partial (V, A', b)$ through h_ϕ ; Adam step
 - 10: **end for**
 - 11: select the epoch with the lowest validation loss (label-free)
-

5 Experiments

5.1 Case studies

AC unit commitment. IEEE case118 (118 buses, 186 branches, 54 committable units) and case300 (300 buses, 411 branches, 69 units) from MATPOWER [31] via pandapower [28], with quadratic generation costs, a no-load cost of 15% of each unit’s full-output cost, a 10% spinning reserve, branch limits, and voltage bounds $[0.94, 1.06]$ (case118) and $[0.90, 1.10]$ (case300). The parameter $\xi = (p_d, q_d)$ is the nodal load. One system level $s \sim U[0.55, 1.30]$ is drawn per instance, scaled by per-load jitter in $[0.95, 1.05]$ (real) and $[0.90, 1.10]$ (reactive). The network input has dimension 236 and 600. A narrower band $s \sim U[0.80, 1.15]$ is nearly degenerate: a fixed majority commitment is within 0.35% of the reference on case118, so it cannot separate methods. The wide band yields 69 distinct reference commitments among 240 case118 instances and 114 among 268 for case300. The reference is the QCAC iterative method [10] with binaries, solved by Gurobi [19] (MIQCQP per iteration, MIP gap 10^{-3} , $\rho_0 = 10^3$, $\rho \leftarrow 4\rho$, stop at slack $\leq 10^{-3}$, at most 20 iterations). It converged on 240/240 (case118) and 227/268 (case300) instances. QCAC is an *inner* convex approximation, so its solutions are AC-feasible upper bounds, *not* certified global optima. We therefore report *deviation from QCAC*, which may be negative.

Hybrid-vehicle energy management. A convex MINLP version of the hybrid-vehicle problem of [12], with a quadratic engine cost and quadratic battery losses. Over $T = 30$ steps, choose engine power $P_t^{\text{eng}} \in [0, 2]$, battery power $P_t^{\text{b}} \in [-3, 3]$, state of charge $E_t \in [0, 10]$, and an engine mode $z_t \in \{0, 1, 2, 3\}$, to minimize $\sum_t (\alpha_t (P_t^{\text{eng}})^2 + \beta_t z_t) + \eta(E_{\max} - E_T)$ subject to $E_{t+1} \leq E_t - \tau P_t^{\text{b}} - \gamma (P_t^{\text{b}})^2$, $P_t^{\text{eng}} \leq (P_{\max}/S)z_t$ and $P_t^{\text{b}} + P_t^{\text{eng}} \geq D_t$, with $\tau = 1$, $\gamma = 0.1$, $\eta = 2$, $E_0 \sim U(5, 8)$, $\alpha_t \sim U(1, 3)$, $\beta_t \sim U(0.5, 1.5)$ (all drawn once). The parameter is the demand profile $D \in \mathbb{R}^{30}$, $D_t \sim U(0.5, 2.5)$. Gurobi solves all 260 instances to a 10^{-4} gap (median 0.99 s), and every optimal schedule is distinct. The relaxation has no linearization point, so the network predicts cuts only. They act on $[E; P^{\text{eng}}; P^{\text{b}}; z]$ (width $4T + 1 = 121$).

Splits, networks, training. Every case uses 48 test, 24 validation and 144 training instances, a fixed split, and four seeds. AC unit commitment: an MLP with two hidden layers of 256 SiLU units, $K = 2$ (four cut rows), 271k (case118) and 481k (case300) parameters, Adam with a cosine schedule, batch 32. The learning rate is 2×10^{-3} for 30 epochs (case118) and 5×10^{-4} for 26 epochs (case300), with $\tau = 0.08$, initial depth 0.25 and $d_{\max} = 2$. Vehicle: hidden width 16 (5,970 parameters), $K = 2$, $w_{\text{int}} = 5$, learning rate 2×10^{-3} , 25 epochs. Training takes 4,220 s (case118), 6,226 s (case300) and 1,420 s (vehicle) per seed on one core of an Apple M3 Pro.

Baseline: supervised NN proxy. An MLP of the same width maps ξ to the integer decisions (sigmoid outputs and MSE to u^* for unit commitment; linear outputs and MSE to z^* for the vehicle), trained on the reference solutions of the same 144 training instances. Its output goes through the *same* restoration as ours: the same priority-order rule, the same N_{down} and the same pricing for unit commitment (rounding threshold 0.5), and the same ℓ_1 projection for

Table 3: Held-out test results (48 instances $\times$ 4 seeds). Mean absolute cost gap, mean discrete and continuous errors, median solution time. For AC unit commitment the reference is QCAC iterative (not a proven optimum); for the vehicle it is the global optimum. “Labels” is the reference-solver time needed to build the training targets.

Case	Method	gap (%)	discrete (%)	continuous (%)	time (s)	labels
case118	reference (QCAC iterative)	—	—	—	78.3	—
	supervised NN proxy	0.104	2.44	4.28	3.5	4.0 h
	learned cut (ours)	0.176	4.31	7.51	3.3	none
case300	reference (QCAC iterative)	—	—	—	346.6	—
	supervised NN proxy	0.075	2.30	2.87	31.2	13.2 h
	learned cut (ours)	0.061	2.91	3.46	31.9	none
vehicle	reference (Gurobi, global)	0	0	0	0.99	—
	supervised NN proxy	1.818	12.09	10.27	0.039	147 s
	learned cut (ours)	0.123	2.93	3.03	0.011	none

the vehicle. The proxy therefore receives strictly more information than our method, namely one reference solve per training instance.

Metrics. Following [12]: *discrete error*, the share of generators whose commitment differs from the reference (unit commitment) or $\|z - z^*\|_1 / \|z^*\|_1$ (vehicle); *continuous error*, $\|p_g - p_g^{\text{ref}}\|_1 / \|p_g^{\text{ref}}\|_1$ with p_g^{ref} obtained by pricing the reference commitment with the same routine (unit commitment), or $\|P^{\text{eng}} - P^{\text{eng}\star}\|_1 / \|P^{\text{eng}\star}\|_1$ (vehicle); *cost gap* $|J_{\text{dep}} - J^{\text{ref}}| / |J^{\text{ref}}|$; and per-instance *wall-clock time* for the whole pipeline (forward pass, solve, restoration), single-threaded.

5.2 Results

Table 3 and Figure 1 report the held-out results.

Speed. On every case the learned surrogate solves one conic program per instance instead of a sequence of mixed-integer solves. Median time falls from 78.3 s to 3.3 s on case118 (24 $\times$), from 346.6 s to 31.9 s on case300 (11 $\times$), and from 0.99 s to 0.011 s on the vehicle problem (92 $\times$). On unit commitment most of the remaining time is restoration (pricing candidate commitments), which the proxy pays equally; its median time is 3.5 s and 31.2 s.

Vehicle. The learned cut is within 0.123% of the global optimum on average, with 2.93% discrete and 3.03% continuous error. The supervised proxy, given the optimal schedules and the same ℓ_1 projection, has 1.818% gap, 12.09% discrete and 10.27% continuous error, so the learned cut is 15 $\times$ more accurate in cost. We attribute the gap to the proxy having to regress 30 coupled integer modes from a 30-dimensional demand profile with 144 examples, whereas in our surrogate the energy dynamics are enforced exactly by the relaxation.

AC unit commitment. On case118 the supervised proxy is more accurate than the learned cut on all three metrics: 0.104% against 0.176% mean cost deviation, 2.44% against 4.31% discrete and 4.28% against 7.51% continuous error. Both take about the same time (median 3.5 s against 3.3 s), since restoration dominates. Our mean is driven by a tail of instances; the median deviation is 0.058%. On case300 the learned cut has the lower mean cost deviation (0.061% against 0.075%; signed -0.029% , i.e. on average slightly *cheaper* than the QCAC reference), while the proxy reproduces the reference decisions more closely (2.30% against 2.91% discrete and 2.87% against 3.46% continuous error). Lower cost with a larger decision deviation is what the two objectives predict: the proxy imitates the reference decisions, while our loss contains only cost, and on these systems several commitments have nearly equal cost. The labels the proxy trains on cost 4.0 h (case118) and 13.2 h (case300) of reference-solver time; our method uses none of it.

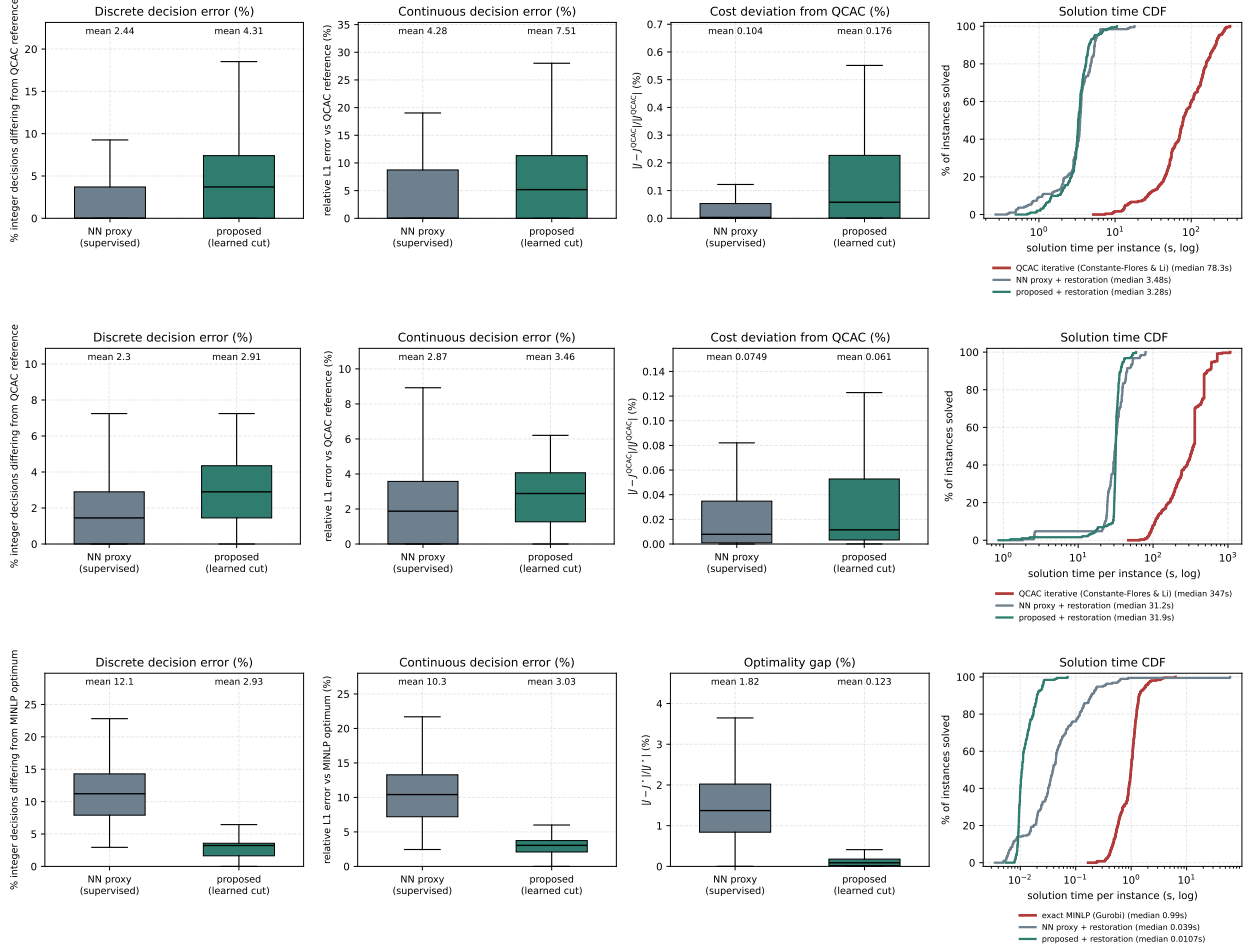


Figure 1: Held-out test results, top to bottom: case118, case300, hybrid vehicle. Box plots of discrete error, continuous error and absolute cost gap (whiskers at 1.5 IQR, outliers hidden) for the supervised NN proxy and the learned cut, and the CDF of per-instance solution time, including the reference solver.

5.3 Gradient quality

Table 4 collects the gradient checks behind the method. Every check compares against central finite differences whose convergence is verified first (agreement between step sizes 10^{-3} and 3×10^{-4}). With an unconverged reference ($h = 10^{-2}$), the correct layer read as 99% wrong. On case118 the exact layer’s gradient of a fixed loss with respect to the cut has cosine 0.9996 with the reference from a single solve plus one sparse LU (0.10 s). A Gaussian-perturbation estimator [6] at the settings used for training ($\sigma = 0.15$, $P = 2$ perturbation pairs, 4 solves) reaches 0.41 ± 0.25 , and the best setting in a 4×6 sweep reaches 0.82 with 64 solves. Larger σ is worse at every sample size (bias), and accuracy grows only like $\sqrt{P}$ (variance). On case300 the gap is wider (0.9995 against 0.18 and at best 0.52). In training, learning the linearization point V with the smoothed estimator made the loss rise on both systems while the exact gradient made it fall.

6 Discussion and limitations

Reference quality. On AC unit commitment the reference is the QCAC iterative heuristic, an upper bound from an inner approximation. Deviations of either sign are deviations from that heuristic, not optimality gaps, and a discrete “error” can be a different commitment of equal or lower cost. *Labels versus no labels.* A supervised proxy and our method answer different questions. The proxy imitates a reference and needs that reference for every training instance. Our

Table 4: Gradient validation against converged finite differences (cosine, or relative error where noted).

Quantity	Case	Agreement
Exact layer, $\partial\ell/\partial b$ (1 solve)	case118 / case300	0.9996 / 0.9995
Smoothed estimator, training setting (4 solves)	case118 / case300	0.41 / 0.18
Smoothed estimator, best of sweep (64 solves)	case118 / case300	0.82 / 0.52
Layer 1 (relaxation w.r.t. cut), held out	case118 / case300	0.944 / 0.969
Layer 2 (priced cost w.r.t. commitment)	case118	0.990
Soft rounding with reserve threshold (8)	case118	1.000000
End-to-end $\partial\ell/\partial b$ through the chain	case118 / case300	0.962 / 0.940
End-to-end $\partial\ell/\partial V$	case118	0.864
Vehicle, $\partial\ell/\partial A$, $\partial\ell/\partial b$ (rel. error, 10 instances)	vehicle	median 0.02% / 0.01%
<code>cvxpylayers</code> on the QCAC relaxation	case118	97% relative error

method optimizes deployed cost directly and needs only parameter samples, so it applies where reference solutions are unavailable or too expensive to generate. Where labels are cheap and the decision map is easy to regress, imitation can be the stronger choice; our case118 results show this. *Restoration*. Both methods rely on restoration to produce integer-feasible solutions, and its design (priority order, repair budget) materially affects the final numbers. Restoration is excluded from the training map, so training optimizes an upper bound on deployed cost. *Training signal on case300*. On case300 the label-free validation loss was lowest within the first five epochs for all four seeds, so the selected models are close to their initialization. The training objective (cost at a soft-rounded commitment) is not yet a faithful enough proxy for deployed cost on this system. *Scale*. Each training step solves two conic programs and two sparse KKT systems per instance on a CPU. Training takes about an hour per seed at case300 size, which is comparable to DFSM’s reported training time but grows with problem size.

7 Conclusion

Learning parametric cutting planes is a natural way to turn a slow mixed-integer solve into a fast convex one while keeping the original constraints. We showed that these cuts can be learned by gradient descent through an exact differentiable conic layer, without any optimal solutions, for MINLPs with second-order-cone relaxations, including nonconvex AC unit commitment, where the network also learns where to convexify. The key practical obstacle was not the idea but the gradient: the relaxations are degenerate, off-the-shelf layers return wrong derivatives, and smoothed estimators are too noisy to train on. With a regularized direct KKT solve and depth-anchored cuts, the exact gradient is accurate and training is stable. The resulting surrogates solve $24\text{--}92\times$ faster than the reference solvers and, on the vehicle problem, are an order of magnitude more accurate than a supervised proxy trained on optimal solutions.

References

- [1] Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond, and J. Zico Kolter. Differentiable convex optimization layers. In *Advances in Neural Information Processing Systems*, 2019.
- [2] Akshay Agrawal, Shane Barratt, Stephen Boyd, Enzo Busseti, and Walaa M. Moursi. Differentiating through a cone program. *Journal of Applied and Numerical Optimization*, 1(2):107–115, 2019.
- [3] Brandon Amos and J. Zico Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning*, 2017.
- [4] Anderson Anrango, André H. Quisaguano, Gonzalo E. Constante, and Can Li. Self-supervised learning of parametric approximation for security-constrained DC-OPF. *arXiv preprint arXiv:2601.13486*, 2026.
- [5] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

- [6] Quentin Berthet, Mathieu Blondel, Olivier Teboul, Marco Cuturi, Jean-Philippe Vert, and Francis Bach. Learning with differentiable perturbed optimizers. In *Advances in Neural Information Processing Systems*, 2020.
- [7] Dimitris Bertsimas and Bartolomeo Stellato. The voice of optimization. *Machine Learning*, 110:249–277, 2021.
- [8] Mathieu Besançon, Joaquim Dias Garcia, Benoît Legat, and Akshay Sharma. Flexible differentiable optimization via model transformations. *arXiv preprint arXiv:2206.06135*, 2023.
- [9] Ján Boldocký, Shahriar Dadras Javan, Martin Gulán, Martin Mönnigmann, and Ján Drgoňa. Learning to solve parametric mixed-integer optimal control problems via differentiable predictive control. *arXiv preprint arXiv:2506.19646*, 2025.
- [10] Gonzalo Constante-Flores and Can Li. A quadratically-constrained convex approximation for the AC optimal power flow. *arXiv preprint arXiv:2501.05623*, 2025.
- [11] Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.
- [12] Shivi Dixit, Rishabh Gupta, and Qi Zhang. Decision-focused surrogate modeling for mixed-integer linear optimization. *Transactions on Machine Learning Research*, 2025.
- [13] Priya L. Donti, David Rolnick, and J. Zico Kolter. DC3: A learning method for optimization with hard constraints. In *International Conference on Learning Representations*, 2021.
- [14] Adam N. Elmachtoub and Paul Grigas. Smart “predict, then optimize”. *Management Science*, 68(1):9–26, 2022.
- [15] Ferdinando Fioretto, Terrence W. K. Mak, and Pascal Van Hentenryck. Predicting AC optimal power flows: Combining deep learning and Lagrangian dual methods. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- [16] Paul J. Goulart and Yuwen Chen. Clarabel: An interior-point solver for conic programs with quadratic objectives. *arXiv preprint arXiv:2405.12762*, 2024.
- [17] Riccardo Grazzi, Massimiliano Pontil, Saverio Salzo, and Alain Zemkoho. Bilevel learning. *arXiv preprint arXiv:2605.01621*, 2026.
- [18] Rishabh Gupta and Qi Zhang. Efficient learning of decision-making models: A penalty block coordinate descent algorithm for data-driven inverse optimization. *Computers & Chemical Engineering*, 170:108123, 2023.
- [19] Gurobi Optimization, LLC. Gurobi optimizer reference manual, 2024.
- [20] Rabih A. Jabr. Radial distribution load flow using conic programming. *IEEE Transactions on Power Systems*, 21(3):1458–1459, 2006.
- [21] James Kotary, Ferdinando Fioretto, Pascal Van Hentenryck, and Bryan Wilder. End-to-end constrained optimization learning: A survey. In *International Joint Conference on Artificial Intelligence*, 2021.
- [22] Shraman Pal and Can Li. DisjunctiveNet: Neural symbolic learning via differentiable convexified optimization layers. *arXiv preprint arXiv:2605.30456*, 2026.
- [23] Seonho Park and Pascal Van Hentenryck. Self-supervised primal-dual learning for constrained optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [24] Max B. Paulus, Giulia Zarpellon, Andreas Krause, Laurent Charlin, and Chris J. Maddison. Learning to cut by looking ahead: Cutting plane selection via imitation learning. In *International Conference on Machine Learning*, 2022.
- [25] Bo Tang and Elias B. Khalil. PyEPO: A PyTorch-based end-to-end predict-then-optimize library for linear and integer programming. *Mathematical Programming Computation*, 2024.
- [26] Bo Tang, Elias B. Khalil, and Ján Drgoňa. Learning to optimize for mixed-integer non-linear programming with feasibility guarantees. *arXiv preprint arXiv:2410.11061*, 2024.

- [27] Yunhao Tang, Shipra Agrawal, and Yuri Faenza. Reinforcement learning for integer programming: Learning to cut. In *International Conference on Machine Learning*, 2020.
- [28] Leon Thurner, Alexander Scheidler, Florian Schäfer, Jan-Hendrik Menke, Julian Dollichon, Friederike Meier, Steffen Meinecke, and Martin Braun. pandapower—an open-source Python tool for convenient modeling, analysis, and optimization of electric power systems. *IEEE Transactions on Power Systems*, 33(6):6510–6521, 2018.
- [29] Marin Vlastelica Pogančić, Anselm Paulus, Vit Musil, Georg Martius, and Michal Rolínek. Differentiation of blackbox combinatorial solvers. In *International Conference on Learning Representations*, 2020.
- [30] Bryan Wilder, Bistra Dilkina, and Milind Tambe. Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.
- [31] Ray D. Zimmerman, Carlos E. Murillo-Sánchez, and Robert J. Thomas. MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education. *IEEE Transactions on Power Systems*, 26(1):12–19, 2011.

A Implementation details

Relaxation as an SOCP. Handed to Gurobi, the continuous QCAC relaxation of case118 is treated as nonconvex: Gurobi does not certify the rotated-cone form of the rank-one strengthening and runs spatial branch-and-bound, hitting a 120 s limit. The same model is an SOCP and Clarabel solves it in 0.06–0.10 s with an identical objective. This is what makes training feasible: each step solves two conic programs per instance. The SOC strengthening is necessary. Without it the slack still vanishes but the voltage products leave the rank-one manifold, and the relaxation returns a cost below the proven optimum on a test instance.

Layer 2 failures. A soft commitment that only just meets the reserve can make the priced AC problem infeasible. During training we escalate the reserve margin (unit commitment) or lower the threshold (vehicle) and retry, the differentiable analogue of upward repair.

Regularization constants. $\varepsilon = 10^{-8}$ (layer 1) and 10^{-10} (layer 2) at $\rho = 10^6$, each chosen on a validation split of instances. Sharing one value between the layers cut layer 2’s gradient cosine from 0.9999 to 0.42, so the constant is per layer.

Code. The released code (cone layer, both case studies, trained AC unit-commitment checkpoints, reference pools and all result files) reproduces every number in this paper.